Islamic University of Technology (IUT) **Student ID:** **Quiz-2**

Course: CSE 4307 - Database Management Systems **Time:** 30 Min **Marks:** 15

1. Consider tables `Student(ID, Name, Dept)` and `Course(CID, Title, Dept)`. If you run `Student NATURAL JOIN Course`, why might valid records be lost? [1]

   (a) It calculates a Cartesian Product, creating too many rows.
   (b) It equates `Student.Dept` with `Course.Dept`, excluding students taking courses outside their department.
   (c) It requires both tables to have identical Primary Keys.
   (d) Natural Join automatically filters out NULL values in the `Name` column.

   Answer: (b) Correct. Natural Join automatically joins tables on *all* columns with the same name. Here, it joins on `Dept`. This logic assumes a student only takes courses offered by their own department, unintentionally filtering out electives taken in other departments.
   Why others are wrong:
   - (a) A Cartesian Product occurs when there are *no* join conditions (Cross Join).
   - (c) Natural Join matches column *names*, not Primary Keys.
   - (d) Join logic generally ignores NULLs in the *join column*, but it does not filter based on non-join attributes like `Name`.

2. Unlike a standard View, what characteristic defines a **Materialized View**? [1]

   (a) It allows updates to the underlying base tables automatically.
   (b) It is computed "on the fly" every time it is accessed.
   (c) It stores the result of the query.
   (d) It is used only for security and hiding sensitive data.

   Answer: (c) Correct. A Materialized View executes the query once and saves the result as a physical table on the disk. This improves read performance for complex queries but requires refreshing when base data changes.
   Why others are wrong:
   - (a) Updatability depends on complexity (e.g., aggregations usually prevent updates) and is not the defining feature of "Materialized".
   - (b) This describes a standard (virtual) view.
   - (d) Both standard and materialized views can be used for security; this is not unique to materialized views.

3. You have two tables, A and B, where A refers to B and B refers to A (Circular Dependency). How can you insert data into both without violating constraints? [1]

   (a) Delete the Foreign Keys, insert data, then recreate Foreign Keys.
   (b) Use `SET CONSTRAINTS ALL DEFERRED` and insert within a single transaction.
   (c) Insert NULLs for the Foreign Keys first, then Update them later (without transactions).
   (d) Circular dependencies are strictly forbidden in SQL.

   Answer: (b) Correct. By deferring constraints, the database waits until the transaction is `COMMITTED` to check validity. This allows a temporary state where constraints are "broken" during the insertion process, as long as they are valid by the end.
   Why others are wrong:
   - (a) Dropping schema constraints is a dangerous structural change, not a data manipulation strategy.
   - (c) This only works if the Foreign Key columns are nullable. If they are `NOT NULL`, this method fails.
   - (d) SQL standards explicitly support circular references via deferrable constraints.

4. When mapping Generalization using **Method 2** (creating tables only for lower-level entity sets), what is a major specific weakness? [1]

   (a) It requires too many Joins to retrieve data.
   (b) Data validation fails because Foreign Key constraints cannot be enforced on the parent entity.
   (c) It cannot handle Disjoint constraints.
   (d) It requires a "type" attribute in the table.

Answer: (b) Correct. In Method 2, the parent table (e.g., `Person`) does not exist physically; only children (e.g., `Student`, `Teacher`) exist. If another table wants to reference "A Person" (regardless of type), it has no single table to point a Foreign Key to.

Why others are wrong:

- (a) Method 2 actually *reduces* joins because all attributes (inherited + specific) are in one table.
- (c) Method 2 implicitly handles disjointness well (by putting data in only one table).
- (d) A "type" attribute is characteristic of Method 1 (Single Table approach).

5. In an ERD, if the `Course` entity set has a relationship `Prereq` with *itself*, what mandatory concept must be used to distinguish the two participations? [1]

(a) Weak Entities (b) Roles (c) Aggregation (d) Ternary Relationship

Answer: (b) Correct. In a recursive relationship, the same entity set participates twice. "Roles" (labels on the connector lines) are required to clarify which entity is the "Prerequisite" and which is the "Main Course".

Why others are wrong:

- (a) Weak entities are for existence dependency, unrelated to recursion.
- (c) Aggregation treats a relationship as an entity, unrelated to recursion.
- (d) A recursive relationship is usually binary (Degree 2), not Ternary (Degree 3).

6. In PostgreSQL/SQL Standard, which expression correctly adds 7 days to the current date? [1]

(a) `CURRENT_DATE + 7`
(b) `ADD_DATE(NOW(), 7)`
(c) `CURRENT_DATE + INTERVAL '7 days'`
(d) `DATE_ADD('7 days')`

Answer: (c) Correct. PostgreSQL uses the standard SQL `INTERVAL` syntax for date arithmetic, allowing you to specify units (days, months, years).

Why others are wrong:

- (a) While some DBs (like Oracle/SQL Server) allow integer addition, PostgreSQL prefers explicit intervals.
- (b) `ADD_DATE` is MySQL syntax.
- (d) `DATE_ADD` is also MySQL syntax and usually requires two arguments.

7. A Weak Entity set does not have a Primary Key. What combination does it use for unique identification? [1]

(a) Its own Partial Key + The Primary Key of the Strong Entity.
(b) Its own Partial Key + The Partial Key of the Strong Entity.
(c) Only the Primary Key of the Strong Entity.
(d) A generated unique integer ID.

Answer: (a) Correct. A weak entity (e.g., `Section`) is identified by its owner (e.g., `Course_ID`) combined with a discriminator or partial key (e.g., `Section_ID`).

Why others are wrong:

- (b) Strong entities have Primary Keys, not Partial Keys.
- (c) The Strong Entity PK alone is not unique enough (one Course has many Sections).
- (d) While you *can* add a surrogate ID, the theoretical definition of a Weak Entity relies on the identifying relationship.

8. Consider a transaction that performs two updates: 1. Deduct $100 from Account A. 2. Add $100 to Account B. [1]
If the first update succeeds, but the second update violates a `CHECK` constraint (e.g., Account B's balance becomes too high), what happens to the first deduction?

(a) It remains saved in the database, but the second update is rejected.
(b) It is automatically rolled back to ensure Atomicity.
(c) The database administrator must manually reverse it.
(d) It is committed, but marked as "incomplete".

Answer: (b) Correct. Atomicity (the 'A' in ACID) ensures that a transaction is an indivisible unit of work. It is either completely applied or completely discarded. A failure in any part triggers an automatic rollback of the whole transaction.

Why others are wrong:

- (a) This describes a partial commit, which violates ACID properties.
- (c) Modern DBMSs handle this automatically; manual intervention is not required for standard constraint failures.
- (d) There is no "incomplete" state for a committed transaction.

9. Consider an Overlapping Generalization (e.g., a `Person` can be both an `Employee` and a `Student`). If you implement this using the Method "Creating tables *only* for the lower-level entity sets `Employee` and `Student`, with no table for `Person`", what specific data quality problem does the material warn against? [1]

   (a) It becomes impossible to assign a Primary Key to the tables.
   (b) Data Redundancy: Shared attributes (e.g., `Address`) are stored twice for the same person.
   (c) The database cannot enforce the "Overlapping" rule; it strictly forces "Disjoint" behavior.
   (d) It requires a "Junction Table" which significantly slows down `SELECT` queries.

   Answer: (b) Correct. Since there is no common `Person` table, the attributes of Person (like Name, Address) must be duplicated in both the `Student` and `Employee` tables. If a person is both, their address is stored in two places, leading to update anomalies.
   Why others are wrong:
   - (a) Both tables can easily have PKs (e.g., SSN).
   - (c) Method 2 actually supports Overlapping well (by allowing an entry in both tables).
   - (d) A Junction Table is used in Method 3 (Parent + Child tables), not Method 2.

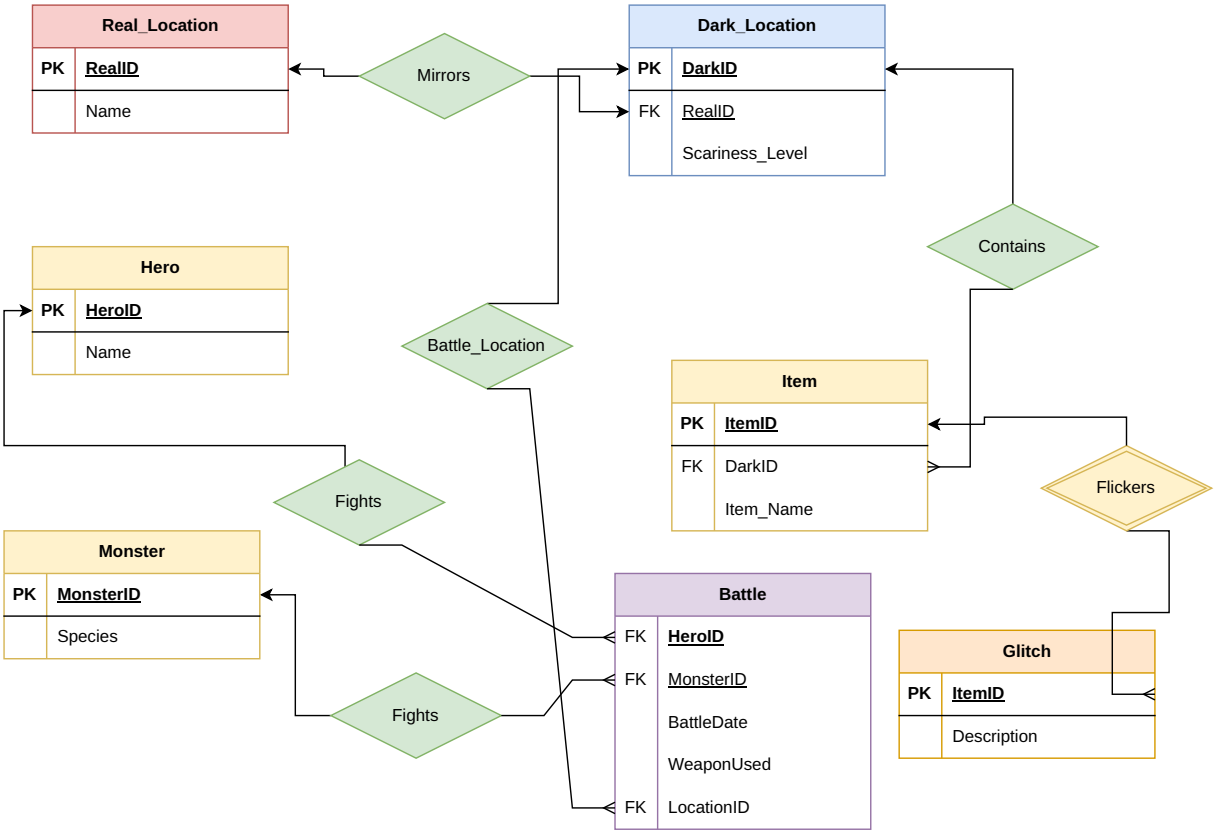
10. You execute: `CREATE ROLE teacher; CREATE ROLE dean; GRANT teacher TO dean;` What does this imply about the privileges? [1]

    (a) The `teacher` role inherits all permissions of `dean`.
    (b) The `dean` role inherits all permissions of `teacher`.
    (c) Both roles lose their previous permissions.
    (d) This is invalid syntax; roles cannot be granted to roles.

    Answer: (b) Correct. The syntax `GRANT A TO B` means "Make B a member of A". Therefore, B (Dean) gains all privileges associated with A (Teacher).
    Why others are wrong:
    - (a) This logic is reversed.
    - (c) Granting roles adds permissions; it does not revoke existing ones.
    - (d) Granting roles to other roles is a standard feature in PostgreSQL (Role-Based Access Control).

11. **The Frozen Timeline:** You have been tasked with designing a database for the 'Upside Down' dimension. The system maps the world as a mirror image: every Real Location (like Hawkins High) connects to exactly one corresponding Dark Location. Inside these Dark Locations, heroes discover various Items; while a single location can hold many items, each item is found in only one specific place. Some of these items are unstable and possess Glitches (visual errors like 'flickering'); a glitch is a dependent entity that cannot exist without the specific item it belongs to. Finally, the system tracks Battles between Heroes and Monsters. Because a hero can fight many monsters and a monster can be fought by many heroes, you must record a log of every specific encounter, including the Date it happened and the Weapon Used. [5]

    Answer:



Some attributes are optional and can vary based on your design